

Programmering med PySpark

Søren Magnusson
programmeringsspecialist, datateknikker

2026-09-01

Indholdsfortegnelse

1 Indledning	1
2 Hentning af datafiler	1
2.1 Chunkning på lokal disk	1
2.2 Eksempel med urllib	3
2.3 Skriv og læs filer på HDFS	3
3 Spark DataFrames	4
3.1 Brug SparkSession	4
3.2 Indlæsning af datafiler	4
3.3 Udlæsning af datafiler	4
4 Kør på spark serveren med spark-submit	4

1 Indledning

2 Hentning af datafiler

Datafiler kan hentes fra internettet med flere biblioteker. F.eks. urllib.request, requests eller wget.

Uanset hvilket library du bruger, er det en udfordring hvis data filerne er store. Man kan ikke bare hente det hele ind i RAM. Datafilerne kan måske ikke få plads på den lokale disk. Derfor er det nødvendigt at hente datafilerne direkte ind i HDFS.

2.1 Chunkning på lokal disk

Et af midlerne til at håndtere store data-filer er at dele problemet op i mindre dele. Altså at læse filen(e) en mindre del ad gangen, og skrive den lille del, og næste. Osv.

Det er klart langsommere på den måde, men når filerne er større end RAM, så er det nødvendigt. Man kan vælge en større eller mindre størrelse af den bid, som man læser og skriver ad gangen. Det er nok realistisk at bruge 1 Megabyte eller mere. Måske 10 MB, eller måske endda mere. Hvis det så er filer som er mindre end det, bliver de snuppet i en bid!

I eksemplet herunder har jeg chunk_size til 1 kb, for at man kan se at selv en mindre fil bliver det i chunks. Men 1 kb er altså en meget lille (nok for lille) størrelse.

[<https://bluebirz.net/posts/make-it-chunks/>] viser hvordan man kan hente store datafiler, der er for store til RAM, i chunks og skrive dem til disk:

```

with open("BigData/src/demo/input_data/iris.csv", 'r') as src:
    with open("BigData/src/demo/input_data/iris_extracted.csv", 'w') as sink:
        chunk_size = 1024 # 1 kb
        while True:
            chunk = src.read(chunk_size)
            if not chunk:
                break
            sink.write(chunk)
            print(chunk)

```

```

$ /home/smag/BigData/src/demo/.venv/bin/python /home/smag/BigData/src/demo/
chunks1.py
5.1,3.5,1.4,0.2,Iris-setosa
4.9,3.0,1.4,0.2,Iris-setosa
...
4.9,3.1,1.5,0.1,Iris-setosa
5.0,3.2,1.2,0.2,Iris-setosa
5.5,3.5,1.3,0.2,
Iris-setosa
4.9,3.1,1.5,0.1,Iris-setosa
4.4,3.0,1.3,0.2,Iris-setosa
...
5.6,2.5,3.9,1.1,Iris-versicolor
5.9,3.2,
4.8,1.8,Iris-versicolor
6.1,2.8,4.0,1.3,Iris-versicolor
6.3,2.5,4.9,1.5,Iris-versicolor
...
5.7,2.8,4.1,1.3,Iris-versicolor
6.3,3.3,6.0,2.5,Iris-virginica
5.8,2.7,5.1,1.9,Iris-virginica
7.1,3.0,5.
9,2.1,Iris-virginica
6.3,2.9,5.6,1.8,Iris-virginica
6.5,3.0,5.8,2.2,Iris-virginica
...
6.3,2.8,5.1,1.5,Iris-virginica
6.1,2.6,5.6,1.4,Iris-virginica
7.7,3.0,6.1
,2.3,Iris-virginica
6.3,3.4,5.6,2.4,Iris-virginica
6.4,3.1,5.5,1.8,Iris-virginica
...
6.2,3.4,5.4,2.3,Iris-virginica
5.9,3.0,5.1,1.8,Iris-virginica

```

Her har jeg forkortet outputet med ... for det ikke skal fylde alt for meget. Bemærk der hvor linierne som ligesom knækker. Det er der hvor chunksne skifter.

2.2 Eksempel med urllib

Vi har lagt op til en diskussion om hvilket library og metode der er bedst til at hente og flytte store datafiler, fra en net kilde til HDFS.

Det er oplagt at bruge requests, som en del af løsningen, men det skulle være eleverne konklusion.

Her er et eksempel på en simpel løsning med et andet library, nemlig urllib.

```
import os
import requests
import subprocess

from hdfs import InsecureClient
from urllib.request import urlopen
```

...

```
def download_and_upload_hdfs(download_url, hdfs_path, hdfs_namenode='http://localhost:9870', chunk_size=1024*1024):
    filename = os.path.basename(download_url)
    hdfs_client = InsecureClient(hdfs_namenode)
    with urlopen(download_url) as response:
        with hdfs_client.write(f"{hdfs_path}/{filename}", overwrite=True) as hdfs_file:
            while True:
                chunk = response.read(chunk_size)
                if not chunk:
                    break
                hdfs_file.write(chunk)
    return hdfs_path
```

Et par links til hvordan man “chunker” med requests:

- <https://webscraping.ai/blog/python-requests-guide>
- <https://medium.com/@harshgolu82/streamline-big-data-requesting-data-in-chunks-with-pythons-requests-library-ddc0796b24ff>

2.3 Skriv og læs filer på HDFS

Der flere metoder til at skrive fra python til hdfs.

- fra python kan man udføre `hdfs dfs -put ...` ved at udføre dette som shell kommando.
- man kan bruge et af flere libraries
 - <https://hdfsccli.readthedocs.io/en/latest/quickstart.html>

3 Spark DataFrames

3.1 Brug SparkSession

3.2 Indlæsning af datafiler

3.3 Udlæsning af datafiler

4 Kør på spark serveren med spark-submit